DIGITAL ASSESSMENT — PROGRESS REVIEW

Design and Deploy a Cloud-Based Application Using AWS

ThrottleGate — Distributed API Rate Limiting & Traffic Control Platform

Course	**CSE2025 – AWS Solution Architect**
Faculty	Dr Renita R
Student Name	Girimurugan S
Registration No.	24BCA7698

1. Problem Statement

Modern web platforms and microservice ecosystems are constantly exposed to traffic surges — legitimate spikes during peak hours as well as abusive traffic such as credential-stuffing bots, scrapers, and denial-of-wallet attacks. Without a traffic-control layer, a single misbehaving client can exhaust application resources and degrade service for everyone. Traditional in-process rate limiting is insufficient in distributed deployments: when an API runs on multiple instances, counters kept in local memory drift apart and can be bypassed simply by hitting a different instance.

ThrottleGate addresses this problem by providing a standalone, centrally deployed rate-limiting and traffic-control microservice. Any backend service can query ThrottleGate before processing a request and receive an instantaneous allow/deny decision, backed by distributed state (Valkey/Redis with atomic Lua scripts) so that limits are enforced correctly and race-free across any number of client instances. A live administrative dashboard visualises traffic and throttling in real time, and the whole platform is deployed on AWS with monitoring and alerting.

2. Project Objectives

- **Primary objective:** design, implement, and deploy a cloud-based rate-limiting platform on AWS following AWS Well-Architected principles (security, reliability, performance efficiency, cost optimisation).
- **Build a decision API** (GET /v1/check) that returns instant allow/deny verdicts with Retry-After guidance for rejected clients.
- **Implement three pluggable algorithms** — Token Bucket, Sliding Window Log, and Sliding Window Counter — selected via configuration, all executed atomically as Lua scripts in Valkey/Redis.
- **Enforce tier-based, per-client, per-endpoint limits** (free/pro tiers) with a sensible default fallback, stored in a managed relational database.
- **Ship a real-time admin dashboard** (React + Chart.js) showing requests/sec, allow vs deny ratio, and throughput, refreshing every 5 seconds.
- **Provide observability** through Spring Boot Actuator, Micrometer counters, and AWS CloudWatch custom metrics with alarms.
- **Deploy entirely within (or close to) the AWS Free Tier,** with a billing alarm and a documented teardown procedure to avoid unexpected charges.
- **Expose a drop-in integration library** (Spring Boot starter) so downstream services can adopt ThrottleGate with one dependency and one configuration line.

3. Proposed AWS Services

3.1 Mandatory Services

AWS Service	Role in ThrottleGate	Category
AWS IAM	EC2 instance role for the CloudWatch agent with least-privilege policies; separate deployment user; no long-lived keys on the instance.	Security (mandatory)
Amazon EC2 (Compute)	t3.micro (Amazon Linux 2023) running the Spring Boot decision API, nginx serving the React dashboard, and the Valkey in-memory state store.	Compute (mandatory)
Amazon S3 (Storage)	Holds dashboard build artifacts and deployment bundles; static-hosting fallback option for the React dashboard.	Storage (mandatory)
Amazon RDS (Database)	PostgreSQL (db.t3.micro) stores clients, endpoints, tiers, and rate-limit rule configuration.	Database (mandatory)

3.2 Optional / Supporting Services

AWS Service	Role in ThrottleGate
Amazon CloudWatch	Custom metrics (allowed, denied, requests/sec) scraped from the app; EC2 StatusCheckFailed alarm; monthly billing alarm (~$10 threshold).
Amazon SNS	Email notifications for CloudWatch alarms (billing and instance health).
Amazon VPC	Custom VPC (10.0.0.0/16) with public subnet, route table, and internet gateway.
Security Groups	Firewall rules: SSH (22) restricted to the admin IP; HTTP/HTTPS (80/443) public via nginx; port 5432 restricted to the EC2 instance only.
Elastic IP	Stable public address so the dashboard URL survives instance restarts (free while attached to a running instance).

4. Preliminary AWS Architecture

The figure below shows the planned deployment. A single EC2 t3.micro instance hosts the application tier (nginx + Spring Boot API + Valkey), keeping compute cost minimal; RDS provides the managed configuration database; IAM, S3, CloudWatch, SNS, and the VPC/security-group stack complete the design.

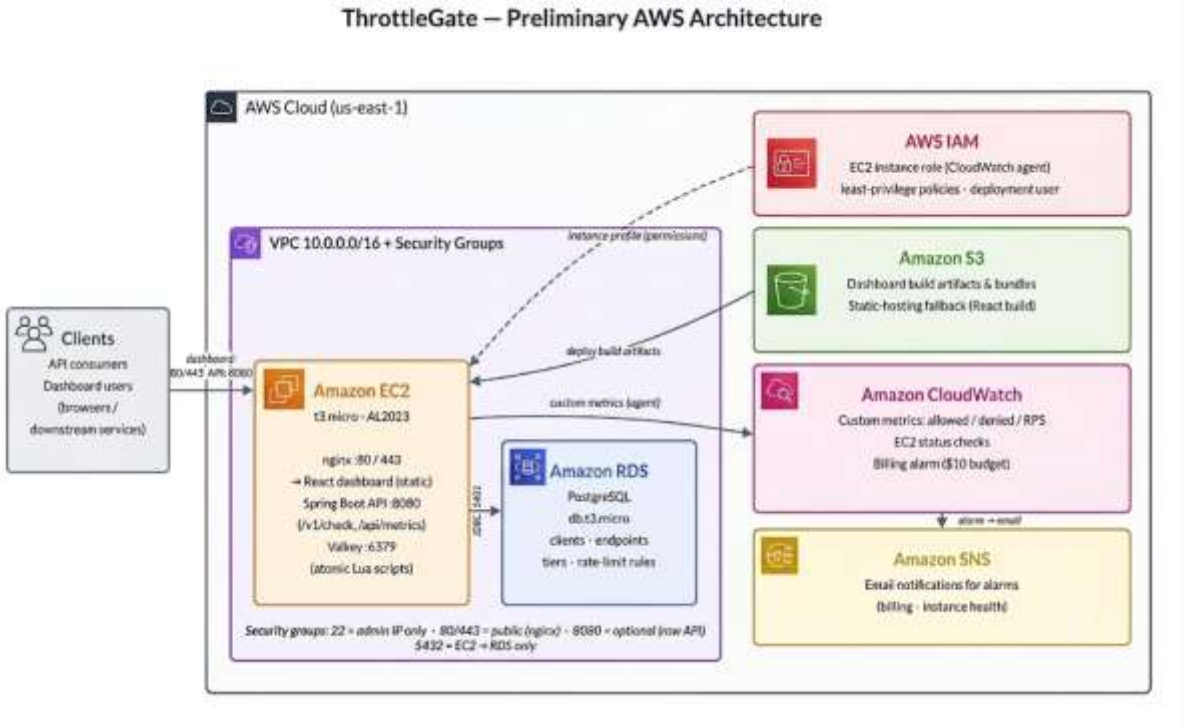


Figure 1 — ThrottleGate preliminary AWS architecture (progress review)

Traffic flow: clients reach nginx (80/443) for the dashboard or the API (8080, optional direct) → the Spring Boot decision API executes a Lua-backed, atomic check in Valkey and consults the tier rules in RDS PostgreSQL → metrics flow to CloudWatch, whose alarms notify via SNS. The EC2 instance role granted through IAM scopes the agent's permissions to CloudWatch only.

5. Current Progress Report

5.1 Completed Work

- **Application built and running locally:** the complete ThrottleGate platform (core service, dashboard, starter library) is implemented in Java 17 / Spring Boot 3 with a React 18 dashboard, verified end-to-end via Docker Compose (API :8080, PostgreSQL :5432, Valkey/Redis :6379).
- **Decision API verified:** GET /v1/check returns allow/deny with a Retry-After header on 429; enforcement confirmed by hammering a 10-request endpoint (ten 200s followed by 429s).

- **Three algorithms implemented atomically:** token bucket, sliding-window log, and sliding-window counter run as Lua scripts in Valkey, eliminating race conditions across instances.
- **Metrics pipeline working:** /api/metrics/throttlegate.requests feeds the dashboard (RPS, allowed, denied, allow ratio); Micrometer + Actuator expose Prometheus-format metrics.
- **Live dashboard:** React + Tailwind + Chart.js admin UI auto-refreshing every 5 seconds, with configurable API URL (REACT_APP_API_URL).
- **Security hardening:** dependency vulnerabilities remediated; container secrets moved to environment variables; CORS restricted to the dashboard origin; Spring Cloud Config client disabled by default for standalone boots.
- **Deployment plan finalised:** a phase-by-phase AWS Free Tier deployment plan (account/budget setup, infrastructure, bootstrap script, systemd service, smoke tests, monitoring, teardown) is documented and ready to execute.

5.2 In Progress / Next Steps

- **AWS account and budget setup** — enable Cost Explorer, create a $10 monthly budget with 50/80/100% email alerts (Phase 0 of the plan).
- **Provision infrastructure** — VPC/subnet, security groups, EC2 t3.micro, Elastic IP, RDS PostgreSQL, S3 bucket, IAM role, SNS topic, CloudWatch billing alarm (Phase 2).
- **Bootstrap and deploy** — run the user-data script (Java 17, Valkey, nginx), ship the JAR, register the systemd unit, build and upload the dashboard to nginx (Phases 3–5).
- **Monitoring and final review** — CloudWatch custom metrics and StatusCheckFailed alarm; run smoke tests against the public endpoint; capture evidence for the final evaluation (Phases 6–7).

5.3 Risks and Mitigations

Risk	Mitigation
EC2 t3.micro has 1 GB RAM while running app + Valkey + nginx	Cap JVM heap (-Xmx256m, SerialGC); Valkey maxmemory bounded; stop the instance when not demoing.
Unexpected AWS charges	Billing alarm + budget alerts; stop/terminate resources after the project; Free-Tier-only service choices; documented teardown (terminate EC2, release EIP, delete snapshots).
Valkey/Redis outage could block all traffic	Fail-open default behaviour so legitimate traffic is not denied when the state store is unavailable.
Public exposure of the raw API port	Prefer nginx reverse proxy on 80/443; keep 8080 closed unless external clients need direct access.

Summary: the application is feature-complete and validated locally; all AWS design decisions (service selection, Free Tier strategy, security posture, monitoring, and teardown) are finalised

in the deployment plan. Remaining work is the phased execution of that plan on the AWS account.